

Vibe coding raises the floor - everyone can build software. Agentic engineering preserves the professional quality bar while going dramatically faster. The two are different disciplines. The ceiling for skilled agentic engineers is well beyond 10x. "I can't remember the last time I corrected [the AI], and then I was vibe coding."

Software 3.0 - A New Computer

- **1.0:** Write explicit code. **2.0:** Program via datasets + training neural nets - the programming is arranging data and objectives. **3.0:** Prompting - the context window is your lever; the LLM is the interpreter doing computation in digital information space.
- It's not about programming getting faster - it's **general information processing** that's now automatable. New things are possible that couldn't exist before (e.g., auto-generating knowledge-base wikis from raw documents). "Previous code worked over structured data. This works over everything."

Example - OpenClaw Installer

- Old paradigm: complex shell script handling every platform, ballooning in complexity. New: a block of text you paste to your agent. The agent has its own intelligence - it adapts, looks at your environment, debugs in a loop, and just makes it work. "You don't have to precisely spell out every detail."

Example - MenuGen

- Karpathy built a full Vercel app: OCR → image gen → re-render. The 3.0 version: photo to Gemini + NanoBanana → food images rendered directly into the menu pixels. **The entire app is superfluous.** "That app shouldn't exist."

Extrapolation - Neural Nets as Host

- In the '50s-'60s it wasn't obvious: calculators or neural nets? We went calculator. Now neural nets are flipping back — becoming the **host process**; CPUs the co-processor. "Tool use" becomes a historical appendage.

Verifiability - Why AI is Jagged

- LLMs peak where output is **verifiable** (math, code) because labs use RL with verification rewards. Models become "jagged entities that peak in verifiable domains and stagnate elsewhere."
- Capability also spikes where labs **intentionally add data** — chess leapt GPT-3.5→4 because someone added chess data, not from general scaling.
- We are "at the mercy of whatever the labs put into the mix." No manual — explore it.

The jaggedness paradox: State-of-the-art Opus 4.7 can simultaneously refactor a 100K-line codebase and find zero-day vulnerabilities - yet tells you to *walk* 50 meters to a car wash instead of drive. "This is insane."

- If you're **inside** the RL circuits → you fly. If you're **outside** → you struggle. "You have to figure out which circuits you're in for your application."
- If outside → **fine-tune with your own RL environments**. The technology works - pull the lever with diverse data + verifiable rewards.
- There are **valuable verifiable domains the labs haven't focused on** - significant founder opportunity. "I don't want to give away the answer, but there is one domain..."

Vibe Coding vs Agentic Engineering

Vibe Coding (raises the floor)

Everyone can build. Trust the system, don't correct. December 2025 was the clear inflection - agentic coherent workflows started actually working. "I just trusted the system more and more." Side projects folder is "extremely full."

Agentic Engineering (preserves the bar)

Coordinate spiky, stochastic, extremely powerful agents to go faster *without sacrificing quality*. No new vulnerabilities. You're still responsible for your software. An engineering discipline. "How do you do that properly?"

Taste, Judgment & What Humans Still Own

- Agents are **"intern-level entities"** - you own aesthetics, judgment, taste, oversight, and the spec. "You're in charge of the design and development; the engineers are doing the fill-in-the-blanks."
- Design the spec with the agent; delegate implementation. Know fundamentals (tensor storage vs. views) but forget API trivia (keepdims vs. keep_dim).
- AI code is often bloaty, copy-paste heavy, brittle abstractions. "It works but it's really gross." Simplification is outside the RL circuits — "pulling teeth." Not fundamental — labs haven't done it yet.
- MenuGen bug: agent matched Stripe email to Google auth email for credits. Different emails = lost funds. Agents still make weird design errors.

"You can outsource your thinking but you can't outsource your understanding."

The Agent-Native Future

- Everything is still written for humans - docs, settings UIs, DNS config. **"Why are people still telling me what to do? What is the thing I should copy-paste to my agent?"**
- Infrastructure must become agent-native: **sensors + actuators** over the world, data structures legible to LLMs. "Describe it to agents first."
- Litmus test: give a prompt → app is built and deployed on the internet → you never touched anything. "MenuGen's trouble wasn't writing the code - it was deploying on Vercel, configuring DNS, stringing up services."
- Long-term: agent representation for people and organizations. "I'll have my agent talk to your agent to figure out the details of our meetings."

Hiring Must Change

Stop giving puzzles. "Give a really big project - write a Twitter clone, make it secure, deploy it. Then I'm going to use 10 Codex 5.4x to try to break your website. They should not be able to break it." Watch how someone builds and coordinates agents at scale.

Education & Understanding

- Karpathy uses **LLM knowledge bases** to process information: feed articles → wiki auto-builds → ask questions. "Anytime I see a different projection onto information, I feel like I gain insight."
- "I'm becoming a bottleneck of just knowing what we're trying to build, why it's worth doing, how to direct my agents." Understanding is still the constraint - tools that enhance it are incredibly valuable.

Key Quotes

- "10x is not the speedup you gain. People who are very good at this peak a lot more than 10x."
- "Almost everything is automatable."
- "A lot of people experienced AI last year as a ChatGPT-adjacent thing. You really had to look again as of December."